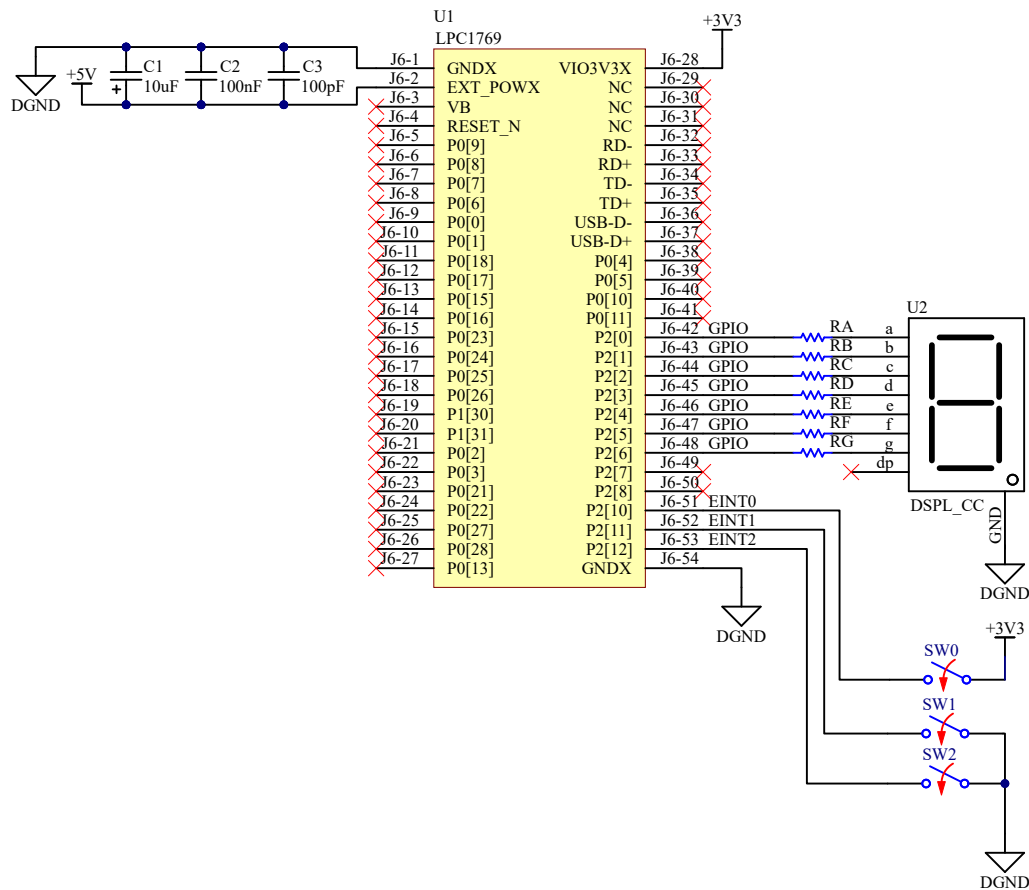


```

1  /*=====
2  * @file    Ejercicio_A_1.c
3  *
4  * @brief    Utilizando SysTick e interrupciones externas escribir un código en C
5  *           que cuente indefinidamente de 0 a 9. Un pulsador conectado a EINT0
6  *           reiniciará la cuenta de 0 y se mantendrá en ese valor mientras el
7  *           pulsador se encuentre presionado. Un pulsador conectado a EINT1
8  *           permitirá detener o continuar la cuenta cada vez que sea presionado.
9  *           Un pulsador conectado a EINT2 permitirá modificar la velocidad de
10 *           incremento del contador. En ese sentido, cada vez que se presione ese
11 *           pulsador el contador pasará a incrementar su cuenta de cada 1 segundo
12 *           a cada 1 milisegundo y viceversa.
13 *
14 * @details  - Salida: P2.0 a P2.6 para los segmentos 'a' hasta 'g'.
15 *           - EINT0 (P2.10): Resetea y congela la cuenta en 0 mientras se presiona.
16 *           - EINT1 (P2.11): Alterna entre pausa y ejecución.
17 *           - EINT2 (P2.12): Alterna la velocidad entre 1 s y 1 ms.
18 *           - Frecuencia de reloj (CCLK): 80 MHz.
19 *
20 * @author   Ing. E. Migliore
21 *
22 * @version  3.0
23  */

```



```

1  /*=====
2  * DIRECTIVAS DE INCLUSIÓN
3  *=====*/
4  #include "LPC17xx.h"
5
6  /*=====
7  * DEFINICIÓN DE CONSTANTES
8  *=====*/
9  #define NVIC_PRIO_BITS      5U          /* Bits físicos de prioridad (0 al 31) */
10
11  /* Identificadores de Prioridad de IRQ */
12  #define EINT0_PRIORITY      //COMPLETAR
13  #define EINT1_PRIORITY      //COMPLETAR
14  #define EINT2_PRIORITY      //COMPLETAR
15  #define SYSTICK_PRIORITY    //COMPLETAR
16
17  /* Identificadores de IRQ */
18  #define SYSTICK_IRQ         //COMPLETAR
19  #define EINT0_IRQ           //COMPLETAR
20  #define EINT1_IRQ           //COMPLETAR
21  #define EINT2_IRQ           //COMPLETAR
22
23  /* Valores de recarga para SysTick */
24  #define N_TICKS_1S          //COMPLETAR
25  #define N_TICKS_1MS         //COMPLETAR
26
27  #define SYSTICK_CTRL_ON     //COMPLETAR
28  #define SYSTICK_CTRL_OFF    //COMPLETAR
29
30  /* Valores de Display */
31  #define DSPL_MAX_DIGITS     //COMPLETAR
32  #define DATA_MASK_DSPL     //COMPLETAR
33
34  /*=====
35  * DEFINICIÓN DE VARIABLES GLOBALES
36  *=====*/
37  const uint32_t lutDSPL_CC[DSPL_MAX_DIGITS] =
38  {
39      //COMPLETAR
40  };
41
42  volatile uint32_t nCurrTicks = N_TICKS_1S;
43  volatile uint32_t cSysTickState = SYSTICK_CTRL_ON;
44  volatile uint8_t index = 0;          //Variable para sincronizar EINT0 y SysTick
45
46  /*=====
47  * PROTOTIPOS DE FUNCIONES
48  *=====*/
49  static inline void cfgNVIC(int32_t, uint32_t);
50
51  void cfgGPIO(void);
52  void cfgIntExt(void);
53  void cfgSysTick(uint32_t, uint32_t);
54  void printDigitDSPL(uint8_t);
55
56  /*=====
57  * FUNCIÓN PRINCIPAL
58  *=====*/
59  int main(void)
60  {
61      /*-----
62      * INICIALIZACIÓN DEL MCU
63      *-----*/
64      cfgGPIO();
65      cfgIntExt();
66      cfgSysTick(nCurrTicks, cSysTickState);
67
68      /*-----
69      * INICIO PROGRAMA PRINCIPAL

```

```

70      *-----*/
71      while(1){}
72
73      return 0;
74  }
75
76  /*=====
77  *  DEFINICIONES DE FUNCIONES
78  *=====*/
79  /*****
80  * @brief   Configura el NVIC y SCB para la habilitación de interrupciones.
81  *
82  * @param   <irq_number>      Número de IRQ en la tabla del vector.
83  * @param   <priority_number> Nivel de prioridad del vector.
84  *
85  * @return  None
86  *****/
87  static inline void cfgNVIC(int32_t irq_number, uint32_t priority_number)
88  {
89      //COMPLETAR
90  }
91
92
93  /*****
94  * @brief   Configura las direcciones GPIO, modos de función y resistencias.
95  *
96  * @details - P2.0 a P2.6: Salidas digitales.
97  *           - P2.10: Función EINT0 con Pull-Down interno.
98  *           - P2.11: Función EINT1 con Pull-Up interno.
99  *           - P2.12: Función EINT2 con Pull-Up interno.
100  *
101  * @param   None
102  *
103  * @return  None
104  *****/
105  void cfgGPIO(void)
106  {
107      //COMPLETAR
108
109      return;
110  }
111
112  /*****
113  * @brief   Configura las interrupciones externas EINT0, EINT1 y EINT2.
114  *
115  * @details - EINT0 (P2.10): Detección por Nivel Alto.
116  *           - EINT1 (P2.11): Detección por Flanco Descendente.
117  *           - EINT2 (P2.12): Detección por Flanco Descendente.
118  *           - Limpia banderas pendientes y asigna prioridades en el NVIC.
119  *
120  * @param   None
121  *
122  * @return  None
123  *****/
124  void cfgIntExt(void)
125  {
126      //COMPLETAR
127  }
128
129  /*****
130  * @brief   Configura el módulo SysTick Timer
131  *
132  * @param   <nTicks> Valor de recarga del registro LOAD.
133  * @param   <cTicks> Registro de control CTRL (Habilitación/Reloj/IRQ).
134  *
135  * @return  None
136  *****/
137  void cfgSysTick(uint32_t nTicks, uint32_t cTicks)
138  {

```

```

139     //COMPLETAR
140 }
141
142 /*****
143  * @brief      Rutina de Servicio de Interrupción (ISR) para el vector EINT0.
144  *
145  * @details    Mientras se mantenga presionado (Nivel Alto), resetea la cuenta a 0.
146  *
147  * @param      None
148  *
149  * @return     None
150  *****/
151 void EINT0_IRQHandler(void)
152 {
153     index = 0;
154     printDigitDSPL(lutDSPL_CC[index]);
155
156     //COMPLETAR
157
158     return;
159 }
160
161 /*****
162  * @brief      Rutina de Servicio de Interrupción (ISR) para el vector EINT1.
163  *
164  * @details    Alterna la habilitación del SysTick Timer (Pausa / Reanudación).
165  *
166  * @param      None
167  *
168  * @return     None
169  *****/
170 void EINT1_IRQHandler(void)
171 {
172     static uint8_t cTicks = 0;
173
174     cTicks++;
175     if (cTicks % 2) {
176         cSysTickState = SYSTICK_CTRL_OFF; // Detiene SysTick
177     } else {
178         cSysTickState = SYSTICK_CTRL_ON;  // Habilita SysTick
179     }
180
181     cfgSysTick(nCurrTicks, cSysTickState);
182
183     //COMPLETAR
184
185     return;
186 }
187
188 /*****
189  * @brief      Rutina de Servicio de Interrupción (ISR) para el vector EINT2.
190  *
191  * @details    Conmuta la velocidad de incremento de desborde del SysTick Timer.
192  *
193  * @param      None
194  *
195  * @return     None
196  *****/
197 void EINT2_IRQHandler(void)
198 {
199     static uint8_t vTicks = 0;
200
201     vTicks++;
202     if (vTicks % 2) {
203         nCurrTicks = N_TICKS_1MS; // Cambia a 1 ms
204     } else {
205         nCurrTicks = N_TICKS_1S;  // Cambia a 1 s
206     }
207

```

```

208     cfgSysTick(nCurrTicks, cSysTickState);
209
210     //COMPLETAR
211
212     return;
213 }
214
215 /*****
216  * @brief    Rutina de Servicio de Interrupción (ISR) para el vector SysTick.
217  *
218  * @details  Incrementa el buffer circular y actualiza el display.
219  *
220  * @param    None
221  *
222  * @return   None
223  *****/
224 void SysTick_Handler(void)
225 {
226     index = (index + 1) % DSPL_MAX_DIGITS; // Incremento en buffer circular
227
228     printDigitDSPL(lutDSPL_CC[index]);
229
230     return;
231 }
232
233 /*****
234  * @brief    Muestra un dígito en el display de 7 segmentos.
235  *
236  * @details  Apaga previamente todos los segmentos y posteriormente establece
237  *           el patrón correspondiente al dígito recibido.
238  *
239  * @param    <digit> Código de segmentos correspondiente al dígito a mostrar.
240  *
241  * @return   None
242  *****/
243 void printDigitDSPL(uint8_t digit)
244 {
245     //COMPLETAR
246
247     return;
248 }

```